Technical Specification: Smart Corridor Intercept (SCI) System

Target Platform: ArduCopter (C++ Firmware)

Objective: Implement a geometric "Shortest Path to Safety" return mechanism that supersedes standard backtracking.

1. Summary

We are implementing a **Smart Corridor Intercept (SCI)** system.

1. **The Inputs:** The drone is loaded with a **Mission** (Spray Path) and a **Safe Corridor** (Return Path) in a single upload.
2. **The Trigger:** Upon an RC switch command or Failsafe, the system calculates the optimal geometric path from the drone's current position on the spray line to the nearest valid entry point on the return corridor.
3. **The Action:** The system dynamically rewrites the active mission to guide the drone to the corridor and then home, skipping unnecessary backtracking.

2. Data Protocol: The "Single KML" Workflow

To ensure compatibility with Mission Planner without custom GCS plugins, we will use a **Separator-Based Protocol** to store both the Spray Path and Return Path in the standard waypoint memory.

2.1 The Mission Format

The flight plan must be uploaded as a single sequence of MAV_CMD_NAV_WAYPOINT items structured as follows:

1. **Phase A: Spray Path (Indices 0….N)**
    - Standard mission waypoints.
2. **Phase B: The Separator (Index N+1)**
    - **Condition:** A Waypoint with Lat = 0, Lon = 0.
    - **Purpose:** Acts as a delimiter. The autopilot must never fly here; it is purely a data marker.
3. **Phase C: Return Corridor (Indices N+2….End)**
    - The sequence of points defining the safe return lane.

2.2 Operational Workflow

- **Pilot:** Imports a KML containing two paths into Mission Planner. (Already done in complete.csv)
- **Pilot:** Inserts a "Dummy Waypoint" (0,0) between the two paths. (Already done in complete.csv)
- **System:** The C++ firmware parses this list on startup or trigger to distinguish the "Active Job" from the "Escape Route."

3. Mathematical Logic (Geodesy & Intersection)

The simulation logic uses 2D Cartesian coordinates. ArduPilot uses Geodetic (Lat/Lon). The implementation must project global coordinates into a Local NED (North-East-Down) frame to perform accurate linear algebra.

3.1 The Projection Step

All calculations are performed relative to the EKF Origin (Home).

$$P_{local} = \text{get_distance_NE}(\text{Origin}_{lat/lon}, \text{Point}_{lat/lon})$$

3.2 The Intersection Algorithm

We iterate through every segment of the **Spray Path** (A to B) and every segment of the **Return Corridor** (C to D) to find where they cross.

Given vectors $r = B - A$ and $s = D - C$:

$$\text{denom} = r_x s_y - r_y s_x$$

If $\text{denom} \neq 0$, we solve for scalars t and u:

$$t = \frac{(C - A) \times s}{\text{denom}}, \quad u = \frac{(C - A) \times r}{\text{denom}}$$

Valid Intersection Condition: $0 \leq t \leq 1$ **AND** $0 \leq u \leq 1$.

3.3 The Directional Decision (Forward vs. Back)

Once an intersection is found at point P_{int}:

1. Calculate $D_{current}$: Distance flown along the spray path to current position.
2. Calculate D_{target}: Distance along the spray path to P_{int}.
3. $\Delta = D_{target} - D_{current}$.
    - If $\Delta > 0$: The intersection is ahead. **Fly Forward.**
    - If $\Delta < 0$: The intersection is behind. **Backtrack.**

4. C++ Implementation Strategy

Recommendation: Implement this as a library **AC_SmartInterceptor** to keep the core code clean.

4.1 Header File (Please see AC_SmartInterceptor.h)

4.2 Source Implementation (Please see AC_SmartInterceptor.cpp)

5. Integration Steps for Developer

1. **Library Creation:** Place the code above into `libraries/AC_SmartInterceptor`.
2. **Compilation:** Add the library to `wscript` or `APM_Config.h`.
3. **Hooks:**
    - Instantiate `AC_SmartInterceptor smart_interceptor;` in `Copter.h`.
    - Map an RC Switch in events.cpp to call `copter.smart_interceptor.trigger_smart_return()`.
4. **Testing (SITL):**
    - Load a mission with the 0,0 separator.
    - Fly the drone past the separator index physically, but keep the active waypoint on the Spray Path.
    - Trigger the switch.
    - **Success Criteria:** The Mission Planner map should immediately update the flight line to cut directly to the pink corridor and exit.